Automated CI/CD Deployment of a Java Web Application Across Four Environments Using GitHub Actions and AWS

Project Overview

This project focuses on automating the deployment of a Java web application across four different environments: Development, Testing, Pre -Production, and Production.

The application source code is maintained in GitHub. GitHub Actions is used to create and automate the CI/CD pipeline. Maven is used as the build tool to compile, test, and package the Java application into a deployable WAR file. Apache Tomcat is used as the application server to deploy and run the application.

The infrastructure is hosted on AWS, where Amazon EC2 instances are used for the application servers and Amazon VPC provides the required network environment. The pipeline automates the process from code changes in GitHub through the build and deployment stages, allowing the application to be promoted systematically from Development → Testing → Pre-Production → Production.

Services Used:

Service / Tool	Purpose
AWS VPC	Provides an isolated and secure network environment for the application infrastructure.
Amazon EC2	Provides virtual servers where the application and Tomcat are installed and run in the four environments.
GitHub	Used to store, manage, and version-control the application source code.
GitHub Actions	Automates the CI/CD pipeline, including build, testing, and deployment activities.
Apache Maven	Used as the build tool to compile the Java application, manage dependencies, run tests, and create the WAR file.
Apache Tomcat	Acts as the application server that deploys and runs the generated WAR file.

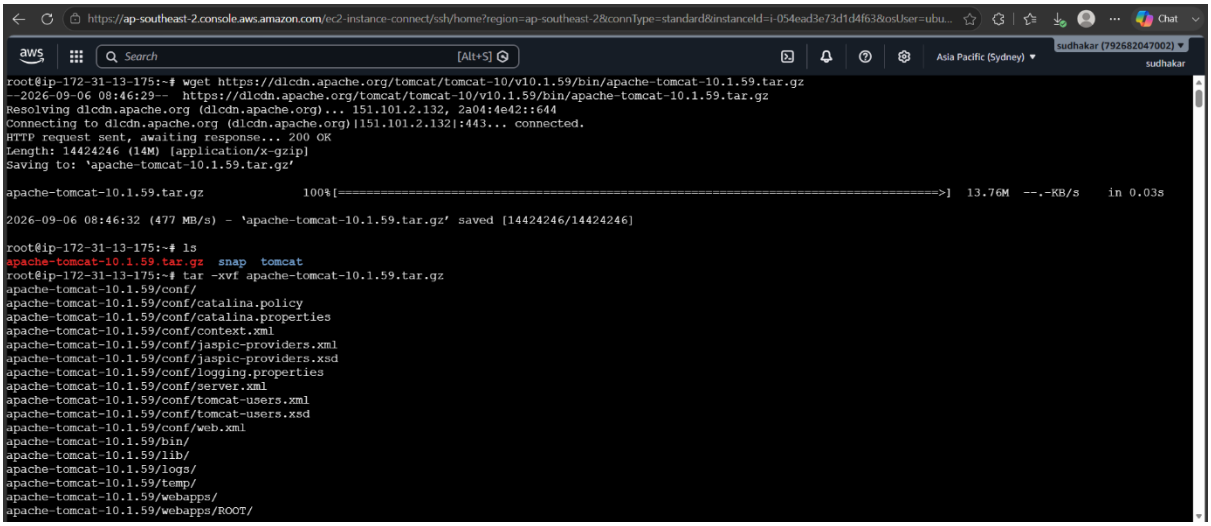
Overall Technology Flow:

Developer → GitHub → GitHub Actions → Maven Build & Test → WAR File → Tomcat → EC2

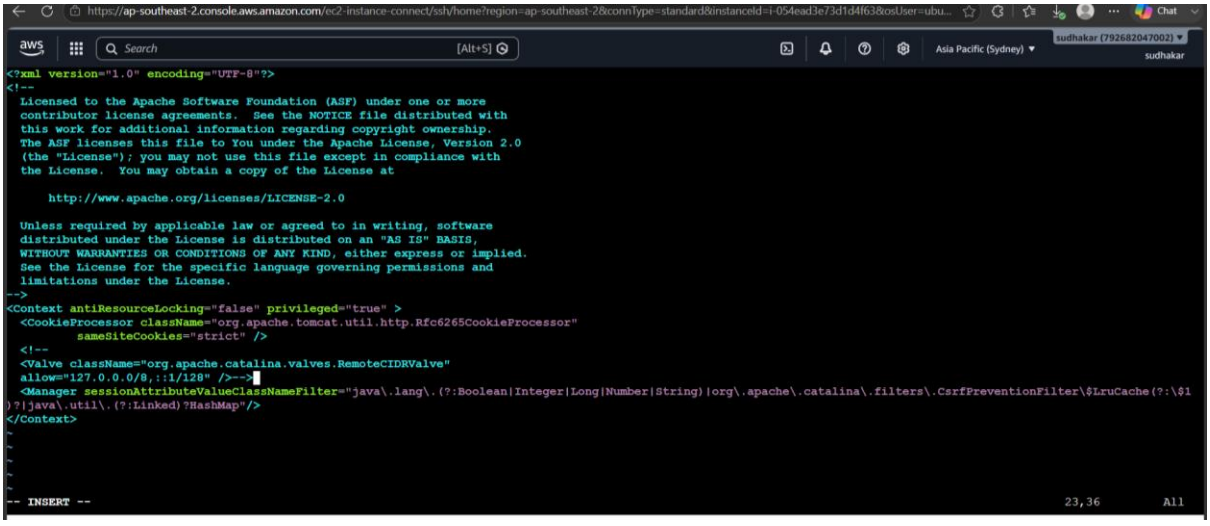
Amazon EC2

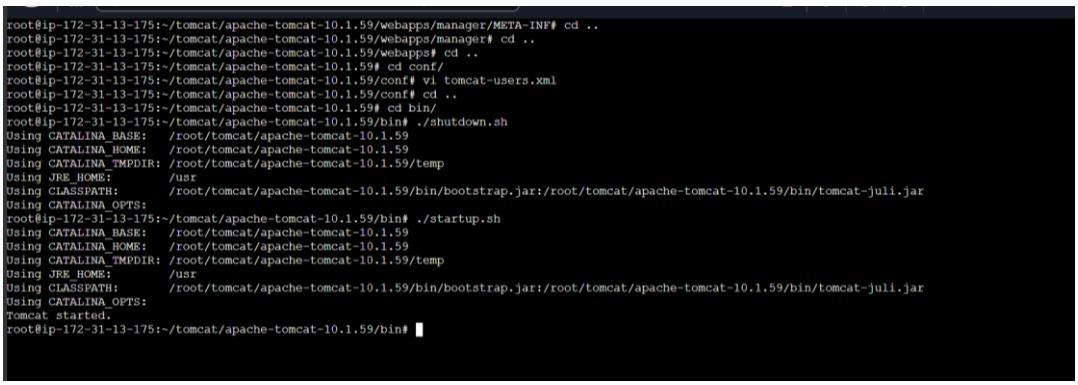
Purpose:
Amazon EC2 (Elastic Compute Cloud) is used to provide virtual servers for hosting and running the Java web application.

In this project, EC2 instances are used for the four environments — Development, Testing, Pre-Production, and Production. The required software, including Java and Apache Tomcat, is installed on the EC2 instances, and the application is deployed and executed through Tomcat.

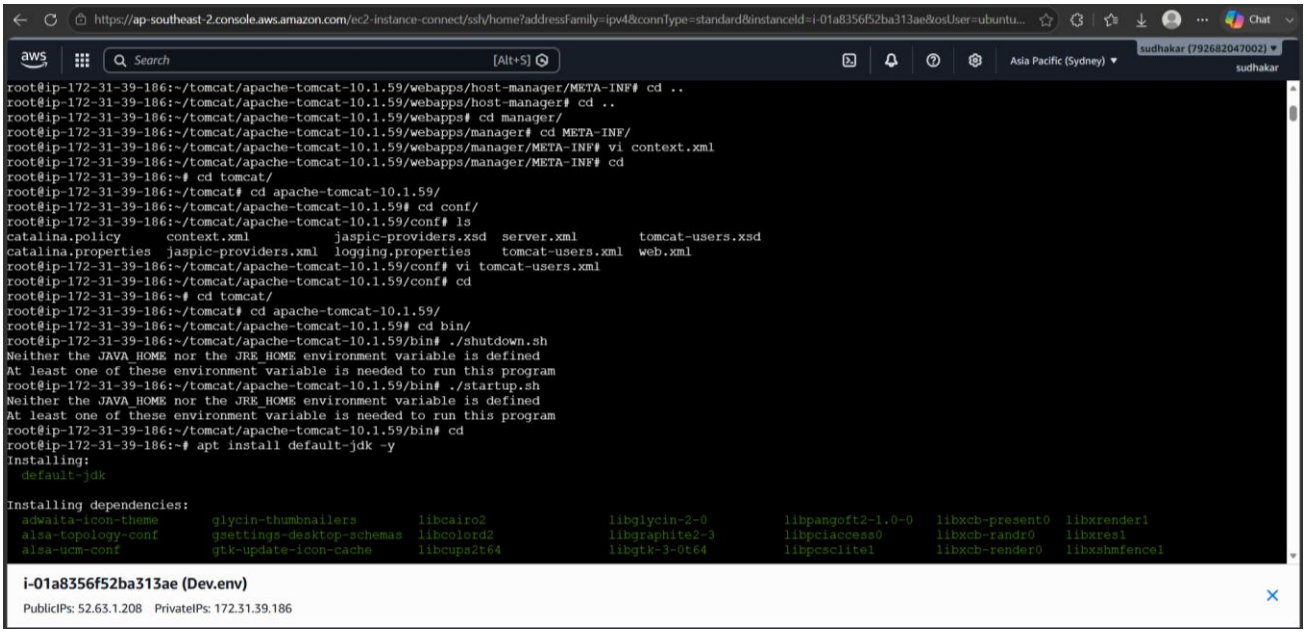


Downloading and unzipping **Apache Tomcat** on the EC2 instances.





- Changed the **ownership and authorization permissions** for the Tomcat **manager** and **host-manager** directories.
- Assigned the required **username, password, and roles** in the Tomcat configuration directory.
- Started the **Apache Tomcat server** after completing the configuration.
- Verified that Tomcat was running successfully.



Installing Java for Application.

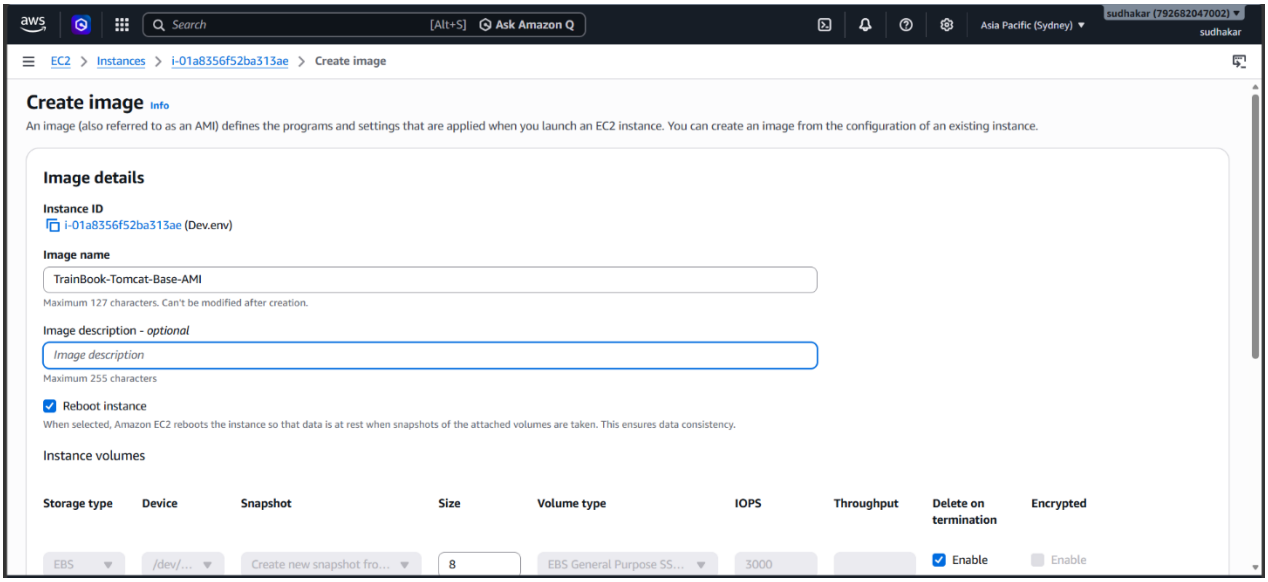
Amazon Machine Image (AMI)

AMI Creation:

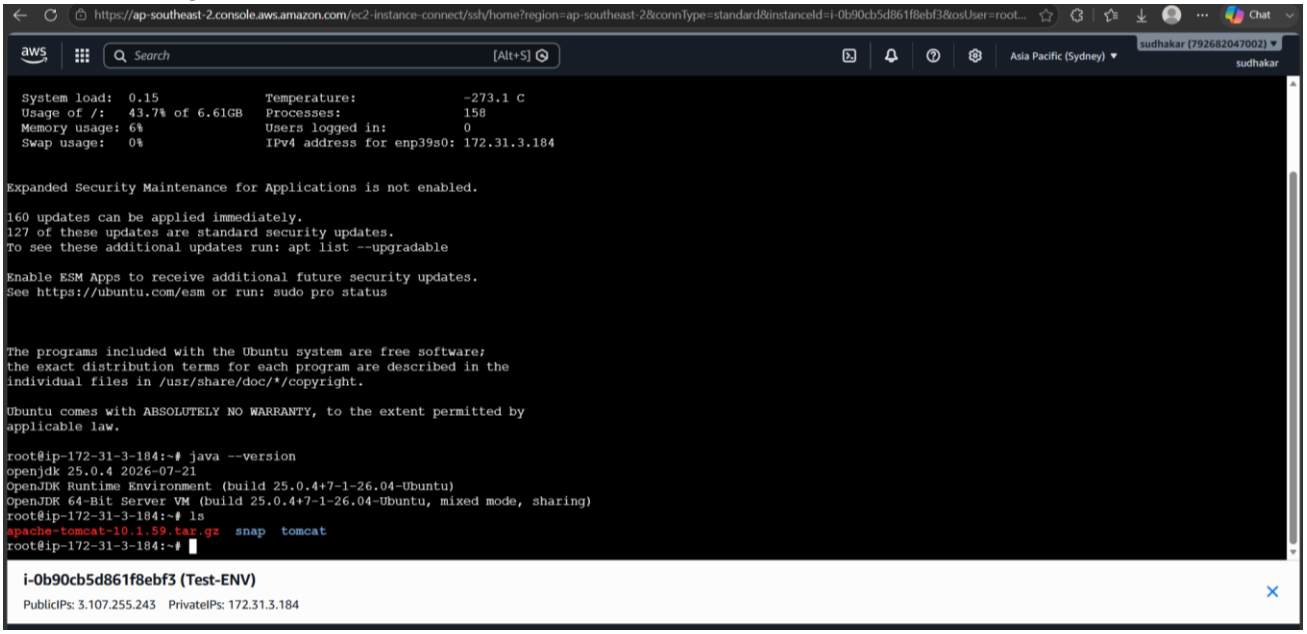
- Created an **AMI from the configured Development EC2 server** after completing the required application and Tomcat configuration.
- The AMI captures the server configuration, installed software, and application setup.
- Used the AMI as a **standardized image to create EC2 instances for the Testing, Pre-Production, and Production environments**.
- This avoids repeating the same Tomcat and application configuration manually on every server.
- Using the same AMI helps maintain **consistent configurations across all four environments**.

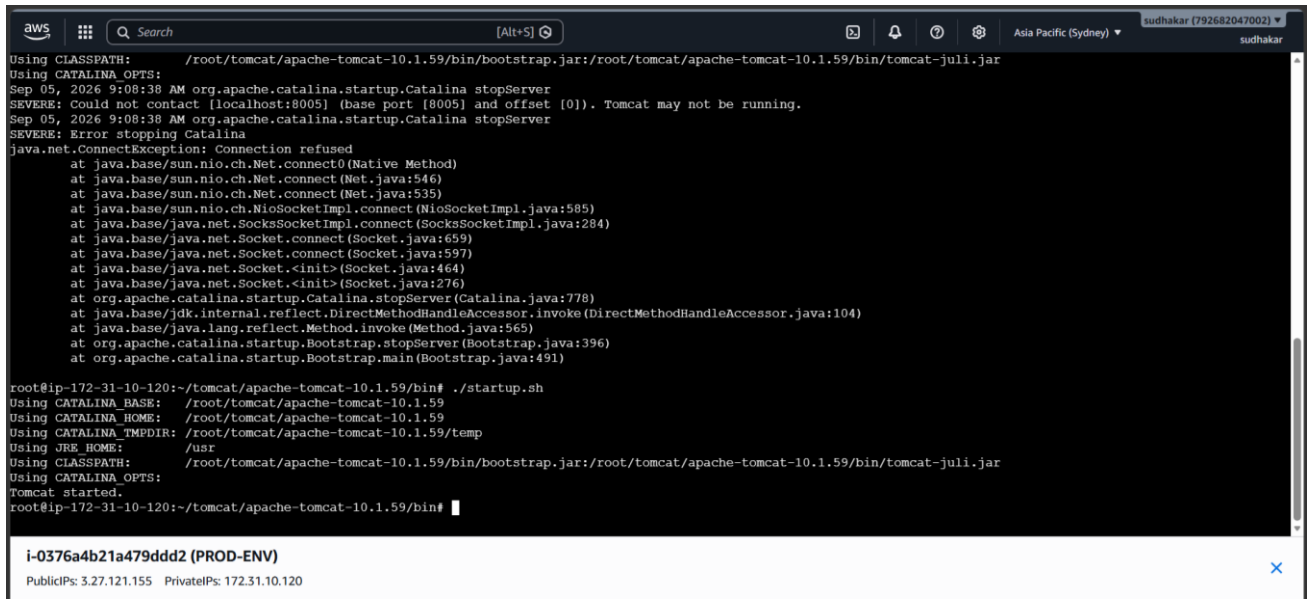
Simple flow:

Development EC2 → Create AMI → Launch EC2 → Testing → Pre-Production → Production



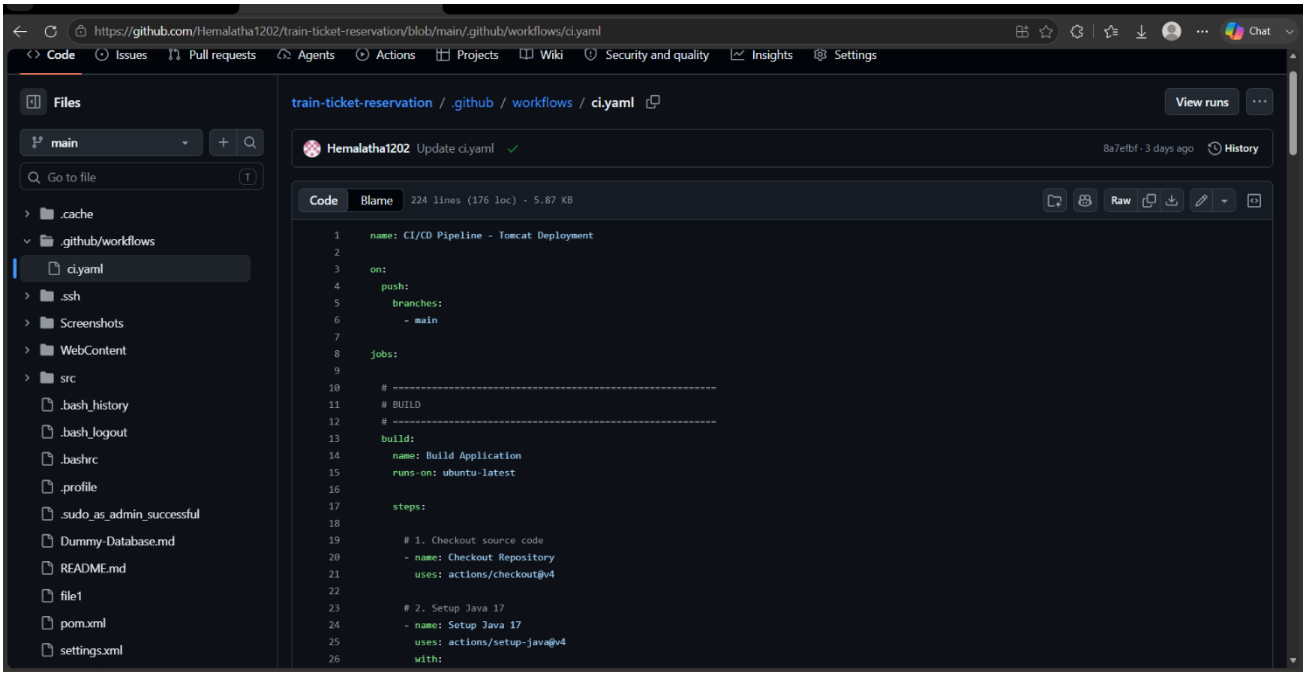
- Creating AMI from DEV-ENV





- Launched a new **Test EC2 server using the AMI created from the Development server**.
- Checked whether all the required software and configurations were successfully copied from the Development server.
- Verified the **Java installation, Apache Tomcat configuration, user roles, permissions, and application setup**.
- Started Tomcat on the Test server and confirmed that the application was running correctly.
- Performed the same verification after launching servers for the **Pre-Production and Production environments**.
- This confirmed that the AMI provided a **consistent server configuration across the different environments**.

Creating the CI/CD YAML File :



CI CD for yml code:

```
name: CI/CD Pipeline - Tomcat Deployment


on:
  push:
    branches:
      - main
jobs:
 # ============================================================
  # BUILD
  # ============================================================
  build:
```

```
name: Build Application
  runs-on: ubuntu-latest

  steps:

    # 1. Checkout source code
    - name: Checkout Repository
      uses: actions/checkout@v4

    # 2. Setup Java 17
    - name: Setup Java 17
      uses: actions/setup-java@v4
      with:
        distribution: temurin
        java-version: '17'
        cache: maven

    # 3. Build WAR
    - name: Build Application
      run: mvn clean package -DskipTests

    # 4. Upload WAR as GitHub Artifact
    - name: Upload WAR
      uses: actions/upload-artifact@v4
      with:
        name: trainbook-war
        path: target/*.war



# ============================================================
# DEV
# ============================================================
deploy-dev:
  name: Deploy to DEV
```

```
needs: build
runs-on: ubuntu-latest

steps:

  - name: Download WAR
    uses: actions/download-artifact@v4
    with:
      name: trainbook-war
      path: war

  - name: Copy WAR to DEV Server
    uses: appleboy/scp-action@v1
    with:
      host: ${{ secrets.DEV_HOST }}
      username: ${{ secrets.DEV_USER }}
      key: ${{ secrets.DEV_SSH_KEY }}
      source: "war/*.war"
      target: "/home/ubuntu/"
      strip_components: 1

  - name: Deploy to DEV Tomcat
    uses: appleboy/ssh-action@v1
    with:
      host: ${{ secrets.DEV_HOST }}
      username: ${{ secrets.DEV_USER }}
      key: ${{ secrets.DEV_SSH_KEY }}
      script: |

        TOMCAT=/root/tomcat/apache-tomcat-10.1.59

        sudo rm -rf $TOMCAT/webapps/TrainBook-Dev
        sudo rm -f $TOMCAT/webapps/TrainBook-Dev.war
```

```
        sudo mv /home/ubuntu/*.war $TOMCAT/webapps/TrainBook-Dev.war

        sudo $TOMCAT/bin/shutdown.sh || true
        sleep 5
        sudo $TOMCAT/bin/startup.sh


# ============================================================
# TEST
# ============================================================
deploy-test:
  name: Deploy to TEST
  needs: deploy-dev
  runs-on: ubuntu-latest

  steps:

    - name: Download WAR
      uses: actions/download-artifact@v4
      with:
        name: trainbook-war
        path: war

    - name: Copy WAR to TEST Server
      uses: appleboy/scp-action@v1
      with:
        host: ${{ secrets.TEST_HOST }}
        username: ${{ secrets.TEST_USER }}
        key: ${{ secrets.TEST_SSH_KEY }}
        source: "war/*.war"
        target: "/home/ubuntu/"
        strip_components: 1

    - name: Deploy to TEST Tomcat
```

```
    uses: appleboy/ssh-action@v1
    with:
      host: ${{ secrets.TEST_HOST }}
      username: ${{ secrets.TEST_USER }}
      key: ${{ secrets.TEST_SSH_KEY }}
      script: |

        TOMCAT=/root/tomcat/apache-tomcat-10.1.59

        sudo rm -rf $TOMCAT/webapps/TrainBook-Test
        sudo rm -f $TOMCAT/webapps/TrainBook-Test.war

        sudo mv /home/ubuntu/*.war $TOMCAT/webapps/TrainBook-Test.war

        sudo $TOMCAT/bin/shutdown.sh || true
        sleep 5
        sudo $TOMCAT/bin/startup.sh


# ============================================================
# PRE-PROD
# ============================================================
deploy-preprod:
  name: Deploy to PRE-PROD
  needs: deploy-test
  runs-on: ubuntu-latest

  steps:

    - name: Download WAR
      uses: actions/download-artifact@v4
      with:
        name: trainbook-war
        path: war
```

```
    - name: Copy WAR to PRE-PROD Server
      uses: appleboy/scp-action@v1
      with:
        host: ${{ secrets.PREPROD_HOST }}
        username: ${{ secrets.PREPROD_USER }}
        key: ${{ secrets.PREPROD_SSH_KEY }}
        source: "war/*.war"
        target: "/home/ubuntu/"
        strip_components: 1

    - name: Deploy to PRE-PROD Tomcat
      uses: appleboy/ssh-action@v1
      with:
        host: ${{ secrets.PREPROD_HOST }}
        username: ${{ secrets.PREPROD_USER }}
        key: ${{ secrets.PREPROD_SSH_KEY }}
        script: |

          TOMCAT=/root/tomcat/apache-tomcat-10.1.59

          sudo rm -rf $TOMCAT/webapps/TrainBook-PreProd
          sudo rm -f $TOMCAT/webapps/TrainBook-PreProd.war

          sudo mv /home/ubuntu/*.war $TOMCAT/webapps/TrainBook-PreProd.war

          sudo $TOMCAT/bin/shutdown.sh || true
          sleep 5
          sudo $TOMCAT/bin/startup.sh


# ============================================================
# PROD
# ============================================================
```

```
deploy-prod:
  name: Deploy to PROD
  needs: deploy-preprod
  runs-on: ubuntu-latest

  steps:

    - name: Download WAR
      uses: actions/download-artifact@v4
      with:
        name: trainbook-war
        path: war

    - name: Copy WAR to PROD Server
      uses: appleboy/scp-action@v1
      with:
        host: ${{ secrets.PROD_HOST }}
        username: ${{ secrets.PROD_USER }}
        key: ${{ secrets.PROD_SSH_KEY }}
        source: "war/*.war"
        target: "/home/ubuntu/"
        strip_components: 1

    - name: Deploy to PROD Tomcat
      uses: appleboy/ssh-action@v1
      with:
        host: ${{ secrets.PROD_HOST }}
        username: ${{ secrets.PROD_USER }}
        key: ${{ secrets.PROD_SSH_KEY }}
        script: |

          TOMCAT=/root/tomcat/apache-tomcat-10.1.59

          sudo rm -rf $TOMCAT/webapps/TrainBook-Prod
```

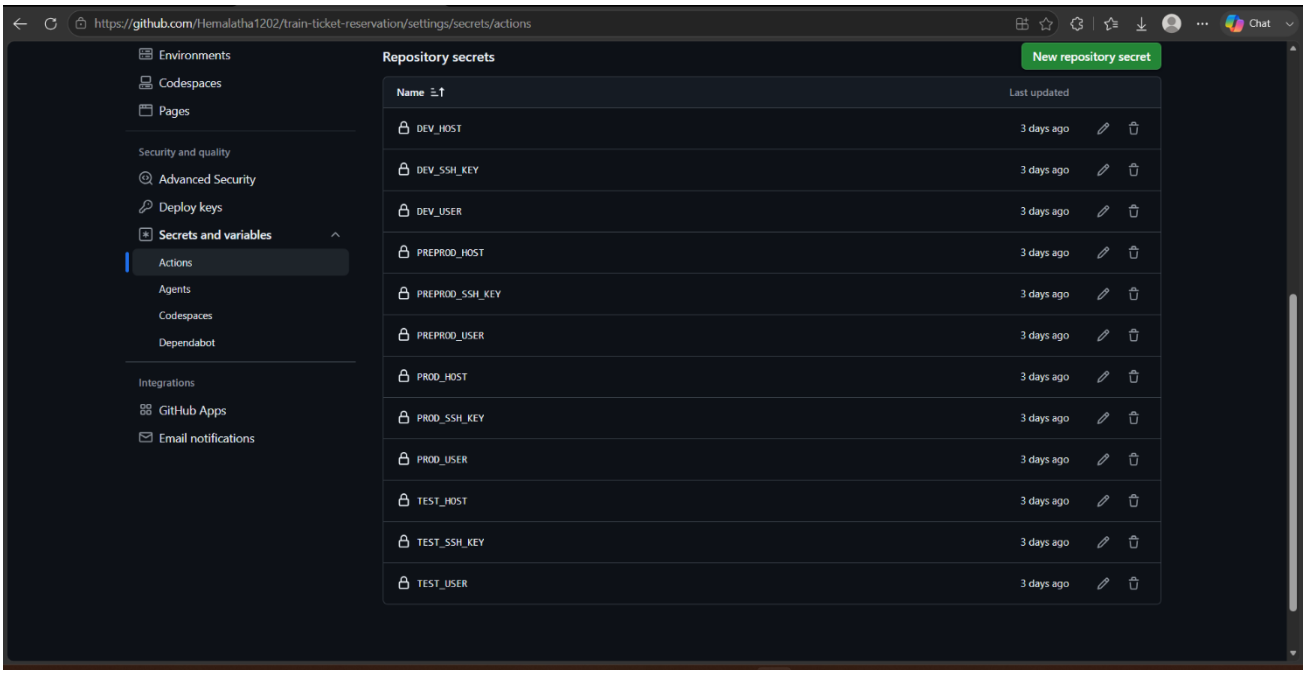
```
sudo rm -f $TOMCAT/webapps/TrainBook-Prod.war

sudo mv /home/ubuntu/*.war $TOMCAT/webapps/TrainBook-Prod.war

sudo $TOMCAT/bin/shutdown.sh || true
sleep 5
sudo $TOMCAT/bin/startup.sh
```

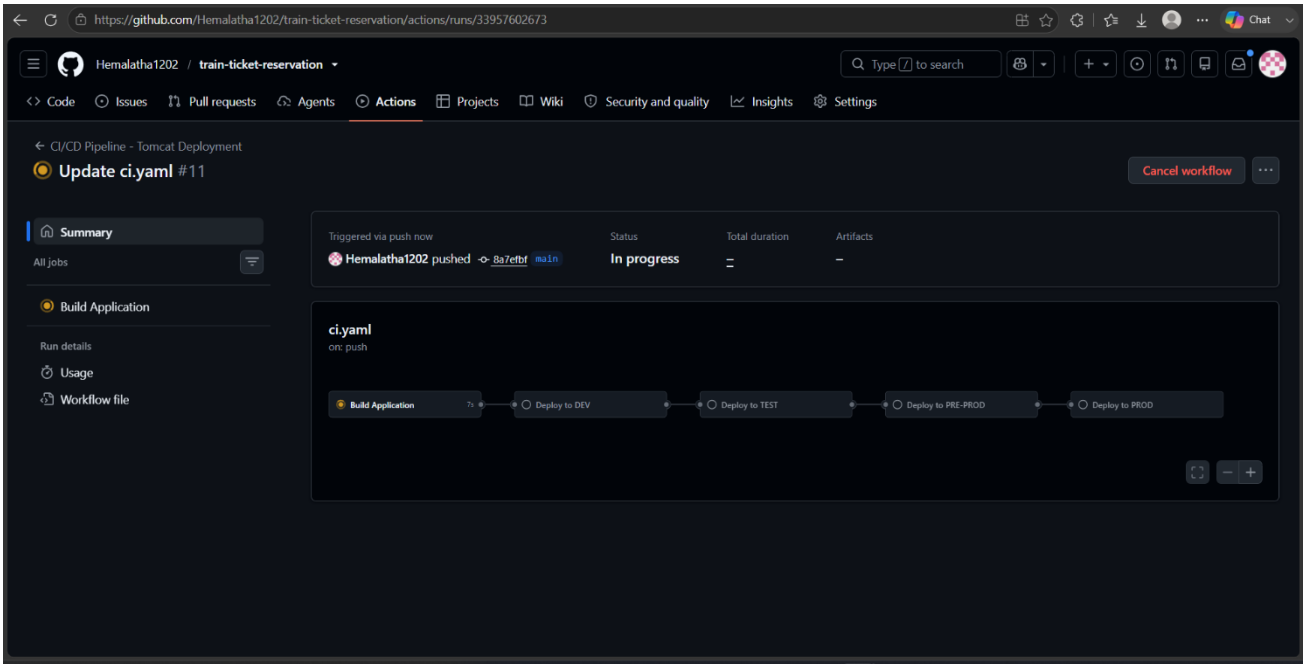
Editing Repository Secrets:

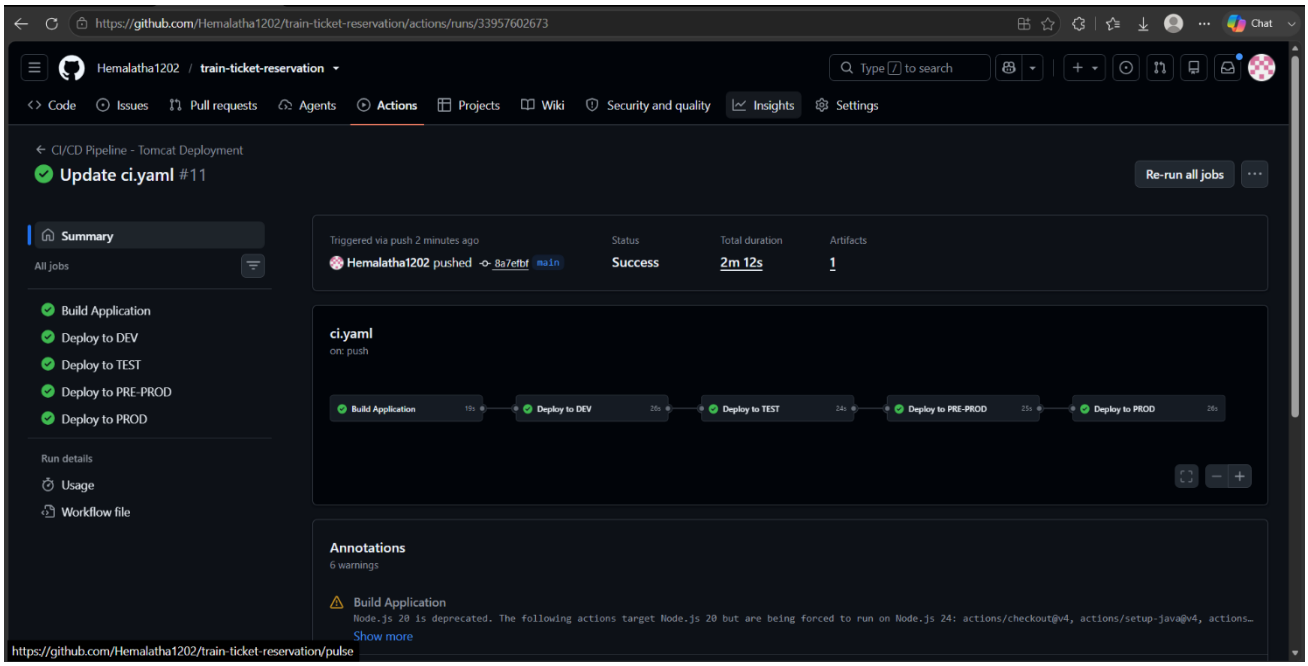
- Added the required **repository secrets** in GitHub to securely store sensitive information used by the CI/CD pipeline.
- Configured the required credentials and server-related values as **GitHub Repository Secrets**.
- Referenced these secrets in the **CI/CD YAML workflow** instead of directly writing sensitive values in the code.



Build and Deployment Process :

- The developer pushes the latest application code to the **GitHub repository**.
- **GitHub Actions** automatically triggers the CI/CD workflow.
- The workflow checks out the latest source code from the repository.
- **Maven** is used to compile the application and run the required tests.
- After a successful build, Maven packages the application into a **WAR file**.
- The generated WAR file is deployed to the **Apache Tomcat server** running on the respective EC2 instance.
- The application is deployed progressively across the four environments:
  Development → Testing → Pre-Production → Production.





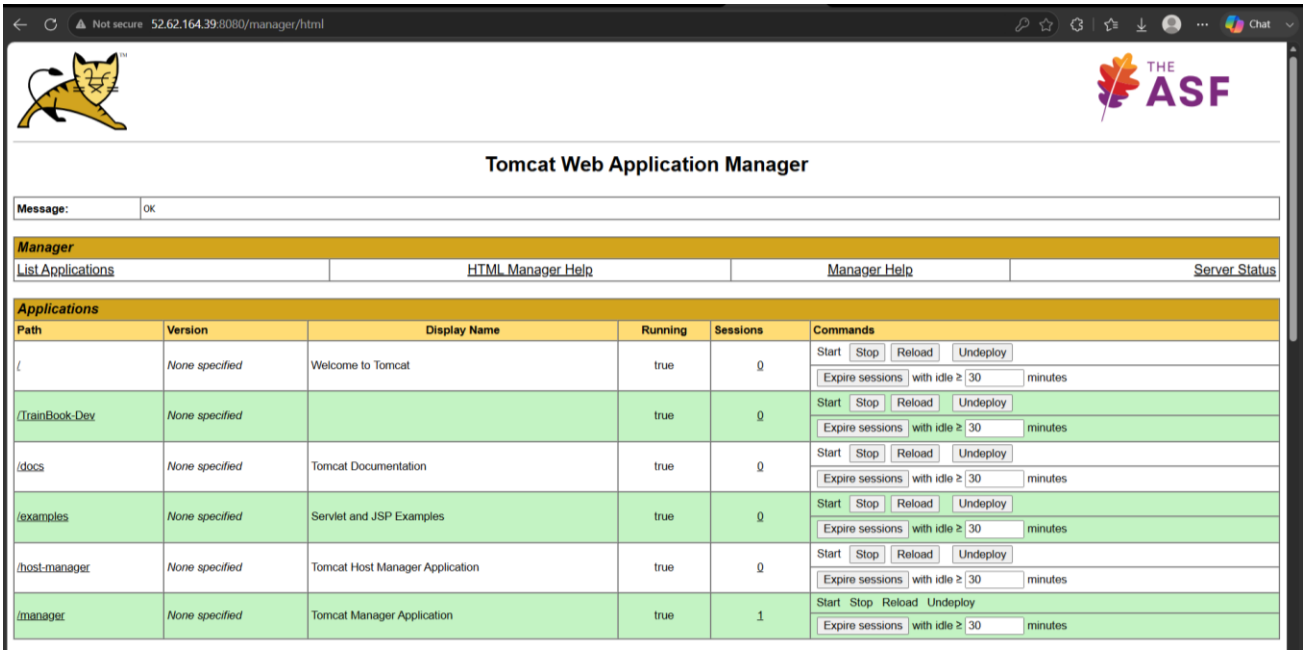
- Triggered the **GitHub Actions CI/CD workflow** by pushing the latest code to the main branch.
- The **Build Application** job executed successfully.
- The application was then deployed sequentially to the **DEV environment**.
- After successful DEV deployment, the pipeline continued to the **TEST environment**.
- The application was then deployed to the **PRE-PROD environment**.
- Finally, the application was deployed to the **PROD environment**.

Environment Mapping:

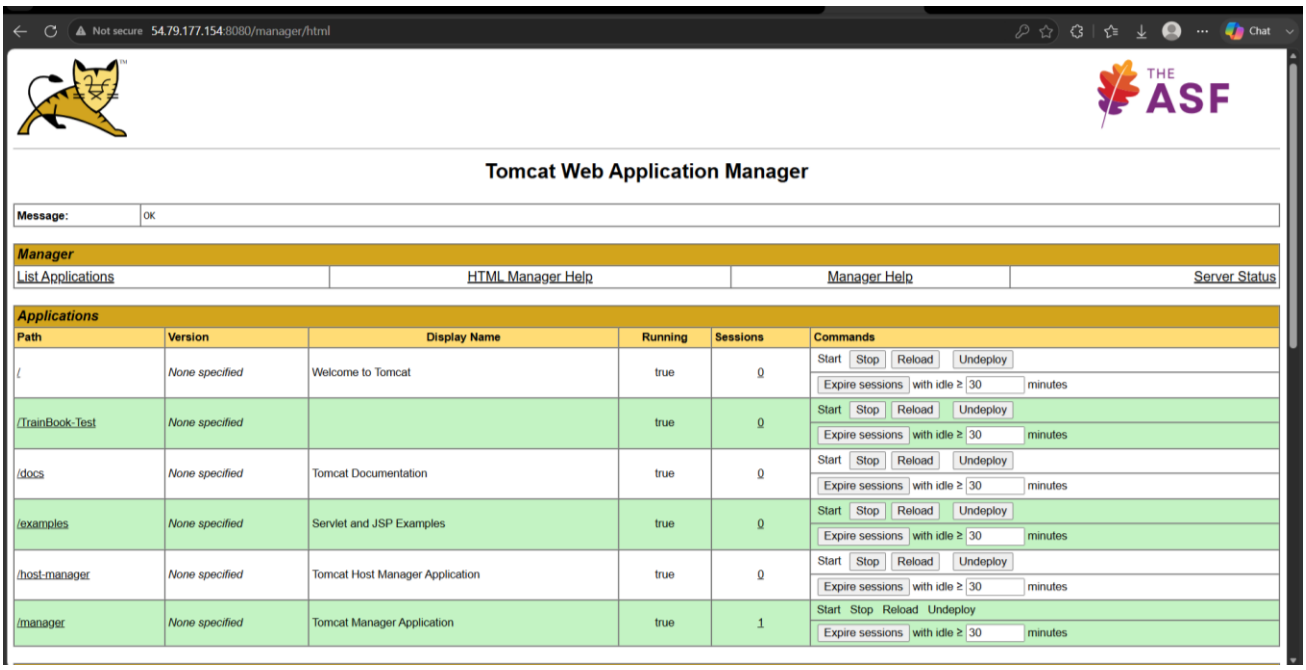
DEV → 52.62.164.39 ec2 IP
TEST → 54.79.177.154 ec2 IP
PRE-PROD → 3.27.121.155 ec2 IP
PROD → 13.211.152.169 ec2 IP

Output of Deployment:

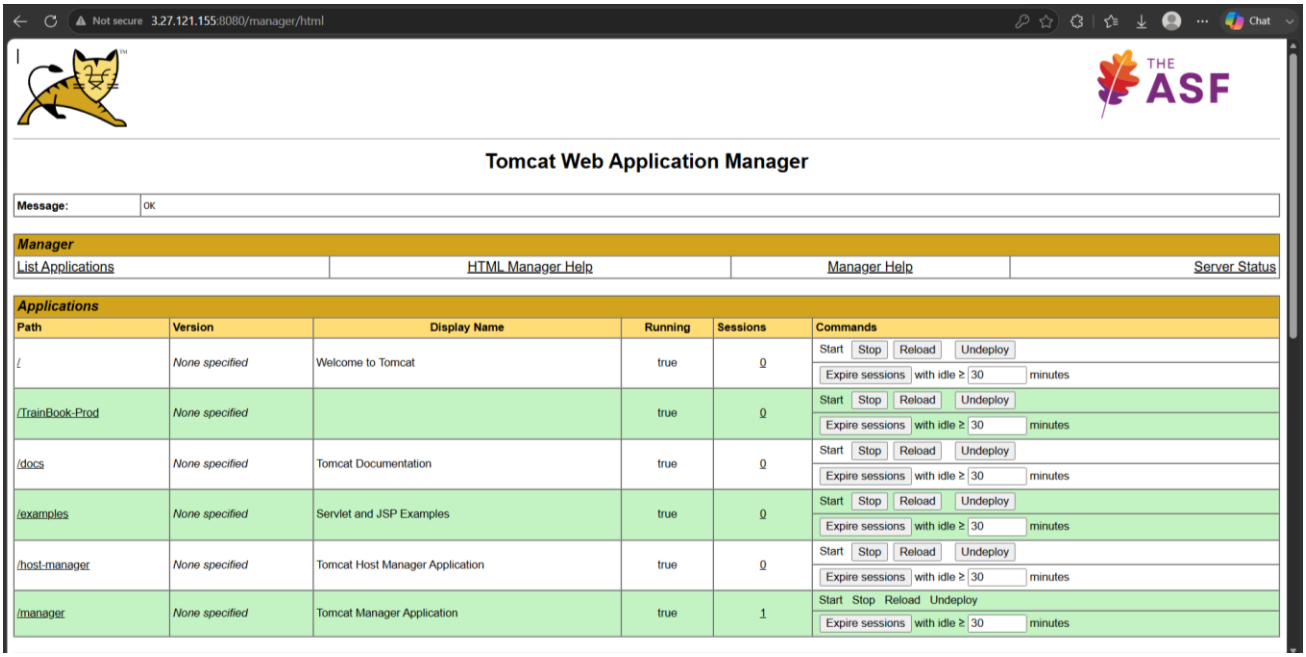
- Successfully deployed the Java web application through the **GitHub Actions CI/CD pipeline**.



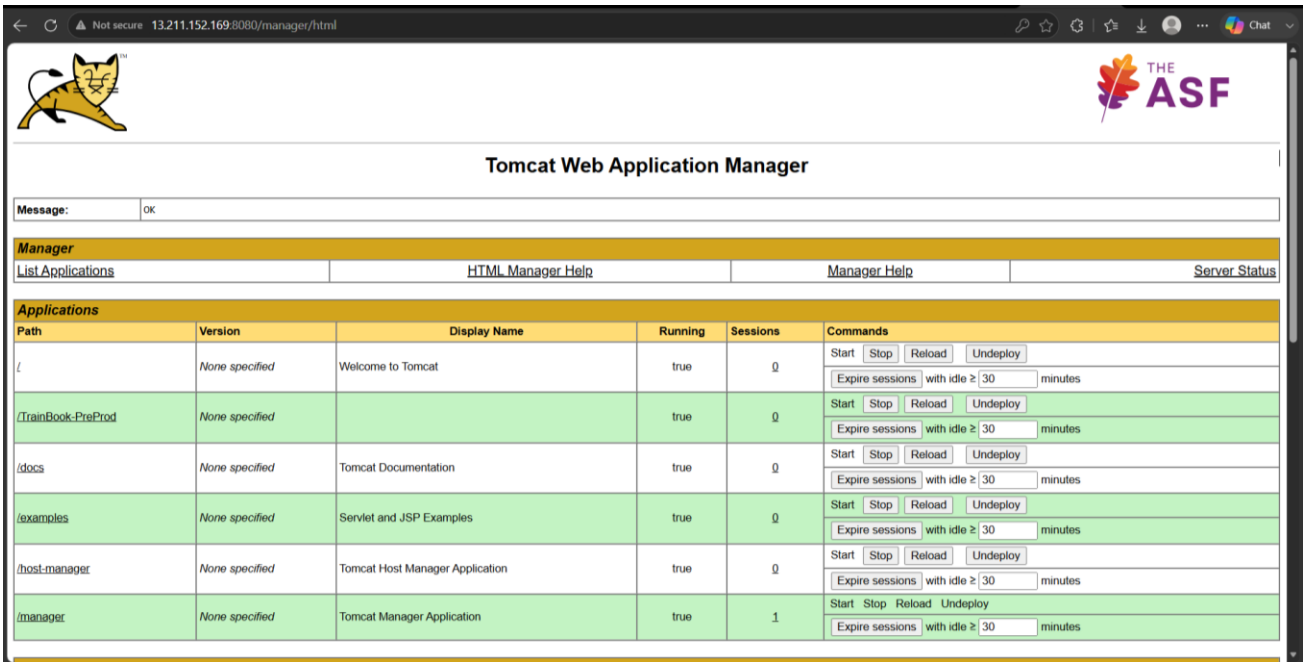
- Output of DEV-ENV



- Output of Test-ENV



- Output of Pre-Prod-ENV



- Output of Prod-ENV